CRYPTO SPREAD MONITOR

Архитектурный дизайн системы

Binance & Bybit · Futures / Spot · Symbol Discovery · SHM Pipeline · Spread Snapshots

1. Обзор системы

Система состоит из четырёх независимых компонентов, взаимодействующих через разделяемую память (POSIX SHM). Каждый компонент делает одну вещь. Наблюдаемость встроена на каждом уровне — задержка измеряется на каждом переходе данных.

Компонент	Роль
symbol_discovery	Запросить REST API каждой биржи, получить все торговые пары по каждому рынку (spot/futures), найти пересечение, сформировать списки подписок для WS
collector	Подписаться на best bid/ask по сформированным спискам символов через WebSocket каждой биржи
normalizer	Привести сырые сообщения бирж к единому формату Quote, записать в SHM-таблицу
spread_reader	Читать SHM, считать спреды по формуле, писать снапшот в текстовый файл

Схема потока данных:

```
symbol_discovery ──► subscription_lists.yaml
                              │
              ┌───────────────┴───────────────┐
     Binance WS (fut+spot)         Bybit WS (fut+spot)
              └───────────────┬───────────────┘
                            collector
                              │  (internal Queue)
                          normalizer
                              │  (POSIX SHM)
                        spread_reader
                              │
                      snapshot_YYYYMMDD.txt
```

2. Список задач к разработке

Ниже перечислено всё, что необходимо разработать. Каждый пункт — отдельная задача. Конкретные решения (алгоритмы, библиотеки, форматы wire-протоколов) выбираются в ходе разработки на основании исследования.

2.1 Symbol Discovery

[ЗАДАЧА] Исследовать: какие REST API endpoints предоставляют Binance и Bybit для получения списка торговых пар по spot и futures рынкам

[ЗАДАЧА] Исследовать: какие поля в ответе позволяют определить активность пары (статус, is_trading, и т.п.)

[ЗАДАЧА] Разработать: модуль запроса торговых пар для каждой биржи и каждого рынка (4 источника: binance_spot, binance_futures, bybit_spot, bybit_futures)

[ЗАДАЧА] Разработать: логику нахождения пересечения — символы, одновременно торгующиеся на spot и futures хотя бы одной пары бирж

[ЗАДАЧА] Разработать: нормализацию символов (привести форматы разных бирж к единому виду для сопоставления)

[ЗАДАЧА] Разработать: формирование итоговых списков подписок — отдельно для каждой биржи и каждого рынка

[ЗАДАЧА] Разработать: сохранение сформированных списков подписок (файл или in-memory структура) для передачи в collector

[ЗАДАЧА] Разработать: логику периодического обновления списков (биржи добавляют/удаляют пары)

2.2 Collectors

[ЗАДАЧА] Исследовать: какие WebSocket channels/streams у Binance и Bybit предоставляют best bid/ask в реальном времени для spot и futures

[ЗАДАЧА] Исследовать: модели подключения — один поток на символ vs один поток на биржу+рынок с мульти-подпиской; выбрать оптимальный подход

[ЗАДАЧА] Исследовать: механизмы keepalive, ping/pong, heartbeat для каждой биржи — они различаются

[ЗАДАЧА] Разработать: абстрактный базовый коллектор с интерфейсом: подключение, подписка, обработка сообщения, переподключение

[ЗАДАЧА] Разработать: 4 конкретных коллектора — Binance Spot, Binance Futures, Bybit Spot, Bybit Futures

[ЗАДАЧА] Разработать: стратегию переподключения с экспоненциальным backoff и логированием каждой попытки

[ЗАДАЧА] Разработать: runner, запускающий все коллекторы согласно спискам подписок из symbol_discovery

[ЗАДАЧА] Определить: как передавать сырые сообщения от коллектора в normalizer (asyncio Queue, pipe, или другое — исследовать)

2.3 Normalizer

[ЗАДАЧА] Разработать: единый датакласс Quote — зафиксировать поля и типы (основа контракта между всеми компонентами)

[ЗАДАЧА] Разработать: парсеры сырых сообщений для каждой биржи и каждого рынка (4 парсера). Для каждого: исследовать wire-формат сообщения и выбрать оптимальный способ извлечения bid/ask и timestamp

[ЗАДАЧА] Разработать: валидацию Quote — проверка на нулевые цены, инвертированный спред, отсутствующие поля

[ЗАДАЧА] Разработать: нормализацию символов из биржевого формата в unified формат

[ЗАДАЧА] Разработать: передачу Quote в shm_writer

2.4 SHM-таблица

[ЗАДАЧА] Разработать: схему разметки памяти — константы: MAGIC, VERSION, HEADER_SIZE, SLOT_SIZE, MAX_SLOTS, offsets каждого поля

[ЗАДАЧА] Разработать: инициализацию SHM-сегмента (создание, запись заголовка, обнуление слотов, проверка MAGIC при рестарте)

[ЗАДАЧА] Разработать: writer — запись Quote в слот с защитой от torn read (seqlock или аналог)

[ЗАДАЧА] Разработать: reader — чтение слота с гарантией консистентности

[ЗАДАЧА] Разработать: маппинг (exchange, market_type, unified_symbol) → slot_id — построение при старте, обновление при появлении нового символа

[ЗАДАЧА] Разработать: cleaner — безопасное удаление SHM-сегмента при остановке или рестарте

[ЗАДАЧА] Определить: поведение при исчерпании MAX_SLOTS — предупреждение без остановки системы

2.5 Spread Reader

[ЗАДАЧА] Разработать: чтение всех активных слотов из SHM

[ЗАДАЧА] Разработать: проверку свежести (staleness) каждой котировки перед расчётом

[ЗАДАЧА] Разработать: расчёт спреда по формуле

[ЗАДАЧА] Разработать: атомарную запись снапшота в текстовый файл (исключить частично записанные файлы)

[ЗАДАЧА] Разработать: runner с заданным интервалом и graceful shutdown

[ЗАДАЧА] Определить: что писать в снапшот для пар с stale данными — N/A или пропускать строку

[ЗАДАЧА] Определить: политику хранения файлов снапшотов — ротация, максимальное число файлов

2.6 Инфраструктура и наблюдаемость

[ЗАДАЧА] Разработать: конфигурационный файл системы (параметры SHM, интервалы, пути к файлам, пороги staleness)

[ЗАДАЧА] Разработать: структурированное логирование — выбрать библиотеку, определить формат (JSON), настроить ротацию

[ЗАДАЧА] Разработать: метрики — определить полный список, типы (counter/gauge/histogram), лейблы

[ЗАДАЧА] Разработать: health check — механизм проверки свежести данных в SHM и доступности коллекторов

[ЗАДАЧА] Разработать: Makefile с командами: init-shm, run, stop, clean-shm, snapshot

[ЗАДАЧА] Составить: requirements.txt / pyproject.toml

3. Symbol Discovery: детальный дизайн

3.1 Назначение

Symbol Discovery — стартовый компонент, без которого система не запускается. Его задача: автоматически определить, по каким символам возможен расчёт спреда (есть котировка и на spot, и на futures), и подготовить списки подписок для каждого WebSocket-коллектора.

Запускается один раз при старте, результат передаётся в collector_runner. Периодически перезапускается для подхвата новых символов.

3.2 Входные данные

4 источника — REST API каждой биржи по каждому рынку:

Источник	Тип	Что нужно получить
Binance Spot	REST	Список всех активных торговых пар
Binance Futures (USDT-M)	REST	Список всех активных контрактов
Bybit Spot	REST	Список всех активных торговых пар
Bybit Futures (Linear)	REST	Список всех активных контрактов

[ПРИМЕЧАНИЕ] Конкретные endpoint URL, структуру ответа и поля активности — исследовать в официальной документации каждой биржи перед разработкой

3.3 Нормализация символов

Биржи используют разные форматы названий торговых пар. Перед поиском пересечения все символы приводятся к единому формату: BASE-QUOTE (например BTC-USDT).

[ЗАДАЧА] Исследовать и задокументировать форматы символов для каждой биржи и каждого рынка

[ЗАДАЧА] Разработать функции конвертации: raw_symbol (per exchange) → unified_symbol

[ЗАДАЧА] Учесть краевые случаи: символы с несколькими разделителями, специфичные суффиксы (PERP, _SWAP и т.п.)

3.4 Поиск пересечения и формирование подписок

Логика поиска общих символов:

1. Собрать unified_symbol множества по каждому источнику: S_binance_spot, S_binance_futures, S_bybit_spot, S_bybit_futures
2. Найти символы, для которых существует хотя бы одна пара (spot_source, futures_source) — т.е. символ есть и в spot, и в futures хотя бы одной из бирж
3. Для каждого такого символа определить, из каких конкретно источников он доступен
4. Сформировать subscription_list для каждого из 4 коллекторов: только символы, доступные в данном источнике

Результирующая структура subscription_lists:

```
subscription_lists:
  binance_spot:    ['BTC-USDT', 'ETH-USDT', ...]   # unified symbols
  binance_futures: ['BTC-USDT', 'ETH-USDT', ...]
  bybit_spot:      ['BTC-USDT', 'ETH-USDT', ...]
  bybit_futures:   ['BTC-USDT', 'ETH-USDT', ...]
```

Collector при подписке конвертирует unified_symbol обратно в raw_symbol нужной биржи используя обратный маппинг.

3.5 Открытые вопросы для проектирования

Вопрос	Варианты	Решить до разработки
Минимальный объём торгов для включения пары?	Без фильтра / по volume / по config	Да
Что делать если REST недоступен при старте?	Аварийная остановка / fallback на кэш	Да

Вопрос	Варианты	Решить до разработки
Как часто обновлять списки подписок?	Раз в час / раз в сутки / по сигналу	Да
Что делать с символом, исчезнувшим с биржи?	Удалить из SHM / помечать stale / игнорировать	Да
Включать только USDT-квотированные пары или все?	Только USDT / из конфига	Да

4. SHM-таблица: детальный дизайн

4.1 Правила работы с SHM

- Один writer (normalizer), множество readers — никаких write-write конфликтов
- Readers никогда не блокируют writer — используем seqlock (sequence counter)
- Каждый слот независим — обновление одного символа не влияет на другие
- SHM имеет фиксированный размер, известный на старте (MAX_SLOTS × SLOT_SIZE + HEADER_SIZE)
- При рестарте: проверить MAGIC, если не совпадает — пересоздать
- POSIX-имя включает версию: /csm_quotes_v1 — для безопасного рестарта при смене схемы

4.2 Инициализация (последовательность)

5. Попытаться открыть существующий сегмент /csm_quotes_v1
6. Если существует — прочитать MAGIC (bytes 0–7): если совпадает — переиспользовать, иначе удалить
7. Создать новый сегмент нужного размера с O_CREAT | O_RDWR
8. Записать заголовок: MAGIC, VERSION, MAX_SLOTS, SLOT_SIZE, created_ts_ns
9. Обнулить все слоты
10. mmap с MAP_SHARED, сохранить указатель

4.3 Разметка памяти

Разработать

4.4 Seqlock — протокол

Writer:

```
slot.seq_begin += 1          # нечётное — идёт запись
<memory barrier>
# записать поля слота
<memory barrier>
slot.seq_end = slot.seq_begin  # чётное — запись завершена
```

Reader:

```
while True:
    seq = slot.seq_begin
    if seq % 2 != 0: continue      # идёт запись — повторить
```

```
    data = read_slot_fields()
    if slot.seq_end == seq: break   # consistent read
```

5. Расчёт спреда и формат снапшота

5.1 Формула

```
spread = (best_ask_futures - best_bid_spot) / best_bid_spot × 100%
```

Правила применения:

- Оба значения должны быть свежими: возраст < STALENESS_THRESHOLD (задаётся в конфиге)
- best_bid_spot > 0 — обязательная проверка
- Если данные stale или отсутствуют — спред не считается, в снапшоте пишется N/A
- Отрицательный спред допустим (бэквордация) — не обрезать

5.2 Формат снапшота

Имя файла: spread_snapshot_<ISO8601>.txt

Запись атомарная: сначала temp-файл, затем os.rename() в финальное имя.

```
# CRYPTO SPREAD MONITOR — Snapshot
# Generated: 2025-01-15T14:32:00.123456Z
# Staleness threshold: 5000ms
#
# SYMBOL        SPREAD%   ASK_FUT   BID_SPOT  FUT_EXCH  SPOT_EXCH  STALE
BTC-USDT        0.1523    43521.50  43454.75  binance   binance    no
ETH-USDT        0.0891    2310.20   2308.14   bybit     binance    no
SOL-USDT        N/A       N/A       N/A       binance   bybit      yes
```

6. Наблюдаемость: метрики и логи Разработать

7. Структура проекта

```
crypto_spread_monitor/
├── symbol_discovery/
│   ├── base_exchange_info.py      # абстрактный запросчик
│   ├── binance_exchange_info.py   # spot + futures
│   ├── bybit_exchange_info.py     # spot + futures
│   ├── symbol_normalizer.py       # raw → unified
│   ├── intersection.py            # логика пересечения
│   └── discovery_runner.py        # оркестратор, периодический запуск
├── collectors/
│   ├── base_collector.py
│   ├── binance_spot_collector.py
│   ├── binance_futures_collector.py
│   ├── bybit_spot_collector.py
│   ├── bybit_futures_collector.py
│   └── collector_runner.py
├── normalizer/
```

```
│   ├── schema.py                  # датакласс Quote
│   └── normalizer.py              # 4 парсера + валидация
├── shm/
│   ├── shm_layout.py
│   ├── shm_init.py
│   ├── shm_writer.py
│   ├── shm_reader.py
│   └── shm_cleaner.py
├── spread_reader/
│   ├── spread_calculator.py
│   ├── snapshot_writer.py
│   └── spread_runner.py
├── infra/
│   ├── logging_config.py
│   ├── metrics.py
│   └── health_check.py
├── config/
│   └── config.yaml
├── cache/                         # кэш symbol discovery
├── snapshots/                     # output
├── logs/
├── requirements.txt
└── Makefile
```